Ders 9 Lab Çalışma Özeti

Ders 9 Lab Çalışma Özeti

Genel Konular

- Prosesler Arası İletişim (IPC - Inter-Process Communication) Giriş
  - Bir işletim sistemi içinde process'ler bağımsız (independent) veya işbirlikçi (cooperating) olabilir.
  - Bağımsız process'ler birbirinden izole çalışır; birbirini etkilemez.
  - İşbirlikçi process'ler birbirinin çalışmasını etkiler; veri paylaşır, haberleşir.
  - İşbirliğinin avantajı: bilgi paylaşımı, computation hızlandırma (paralelleştirme), modülerlik.
  - Örnek: Web tarayıcılar (Chrome) - main process, renderer process, plugin process olarak ayrı çalışır. Bir tab'da hata olursa diğerleri etkilenmez.
- IPC İki Temel Modeli
  - **Shared Memory (Paylaşımlı Bellek)**: İki process aynı fiziksel bellek alanı üzerinden haberleşir. Process'lere ait bellek alanları normalde korunur; ancak shared memory'de her iki process de okuma/yazma yapabilir. OS sadece paylaşılan belleği tahsis eder; haberleşmenin yönetimi process'lere aittir.
  - **Message Passing (Mesajlaşma)**: Process'ler arasında mesaj aktarımı ile haberleşme. Doğrudan veya dolaylı olabilir. Paylaşılan alan yoktur; mesajlar aktarılır.
- Shared Memory ve Message Passing Karşılaştırması
  - Shared memory: Daha hızlı (kernel arada değil, doğrudan bellek erişimi). Haberleşme yönetimi process'lere ait.
  - Message passing: Daha yavaş (kernel dahil), ama distributed sistemlerde de çalışabilir. Mesaj boyutu sınırları, link kapasitesi gibi kısıtlamalar var.
  - Aynı fiziksel sistemde aynı OS üzerinde: shared memory daha hızlı.
  - Farklı sistemlerde: message passing gerekli.
- Producer-Consumer Problemi
  - Temel senkronizasyon problemi. Producer veri üretir, consumer tüketir.
  - Buffer alanı: Bounded buffer (sınırlı) - gerçekçi, gerçek sistemde sınırsız bellek yok.
  - Buffer durumu: in (eklenecek index), out (çıkarılacak index) pointer'ları.
  - Üretici, boş yer yoksa (in == out, buffer dolu) bekler; tüketici, boş yer yoksa (in == out, buffer boş) bekler.
  - Buffer size = 10 örneği verilir.
- Direct vs Indirect Communication
  - **Direct Communication**: Process'ler birbirini açıkça adresler. `send(P, message)` ve `receive(Q, message)`.
  - **Indirect Communication**: Mesajlar mailbox (posta kutusu) üzerinden aktarılır. Process'ler mailbox'ı paylaşır.
- Mailbox (Posta Kutusu) Modeli
  - Her mailbox bir kuyruktur. Mesajlar mailbox'a bırakılır, alıcı mailbox'tan alır.
  - Mailbox kapasitesi: sıfır (randevu) veya sınırlı (buffer).
  - Mesaj önceliği: yüksek öncelikli mesaj önce alınır.
  - "Mailbox'a bırakılan mesaj alındıysa sonsuza kadar alınmış demektir" (tek alıcı varsayımı).
- Buffer Kapasitesi ve Mesaj Boyutu
  - Link kapasitesi sıfır: Mesaj gönderilemez, alıcı bekliyorsa gönderilir (randevu).
  - Link kapasitesi sınırlı: Belirli sayıda mesaj kuyrukta bekleyebilir.
  - Link kapasitesi sınırsız: Ütopik, gerçekte yok.

  - Mesaj boyutu: Sabit (implementasyon kolay) veya değişken (process açısından kolay).

Hocanın Özellikle Vurguladığı Kısımlar

- Shared Memory vs Message Passing Performansı
  - Hoca vurgular: “İkisi de aynı fiziksel sistem üzerinde aynı işletim sistemi üzerinde ise evet zaten hani büyük ihtimalle shared memory çok daha hızlı olacak. Ama zaten farklı sistemlerden bahsediyorsak da yine shared memory’nin kafadan daha iyi olacağını daha hızlı olacağını söyleyebiliriz.”
  - Ancak distributed memory şeklinde birbirine bağlıysa shared memory daha hızlı; uzak sisteme başka türlü erişim şansı yoksa message passing.
- Yönetim Sorumluluğu
  - “Shared memory’deki message passing’e göre önemli farklardan biri kernel işletim sistemi aradaki paylaşılan belleği verdikten sonra haberleşmenin yönetimi işine girmiyor. Bu haberleşmenin yönetimini haberleşen uygulamaların kendisinin yapması lazım.”
  - “Halbuki message passing’te az önce hocanızın da söylediği gibi soruya verilen cevapta da söylendiği gibi öyle veya böyle kernel işin içine dahil oluyor.”
- Producer-Consumer Senkronizasyonu
  - Buffer dolu: Üretici bekler. Boş: Tüketici bekler. in == out ise: Hem dolu hem boş olabilir (count değişkeni ile ayırt edilir).
  - “Çünkü onlar bir önceki turdan diğer filozoflar tarafından alınmış olacak.” (her philosopher durumu)
- Link Kurulumu Kararları
  - İki process arasında kaç link? Birden fazla link olabilir (özellikle uni-directional ise).
  - Link kapasitesi ne olacak? Sıfır, sınırlı, sınırsız.
  - Mesaj boyutu sabit mi değişken mi?
  - Uni-directional (tek yön) mi bidirectional (çift yön) mi?
- Multi-producer / Multi-consumer Yanlış Terim
  - Hoca düzeltir: “Multi producer, multi consumer diye bir terim yok. Producer consumer var. Producer consumer’ın içinde bir tane producer, bir tane consumer; bir tane producer, bir tane consumer; en tane producer, en tane consumer şeklinde kombinasyonları var.”

Kısa Tekrar Notları

- İki temel IPC modeli: Shared Memory ve Message Passing.
- Producer-Consumber problemi: bounded buffer, in/out pointer’ları.
- Direct communication: `send(P, message)`, `receive(Q, message)`.
- Indirect communication: mailbox üzerinden.
- Mailbox: kapasite (sıfır/sınırlı/sınırsız), öncelik.
- Shared memory daha hızlı ama senkronizasyon process’lere ait.
- Message passing daha yavaş ama distributed sistemlerde çalışır.

Detaylı Açıklamalar

Ders 9 Lab, process’ler arası iletişim (IPC) kavramını derinlemesine ele alır. Geçen haftaki süreç yönetimi konusundan sonra, process’lerin birbirleriyle nasıl haberleştiği anlatılır.

IPC’nin neden gerekli olduğu açıklanır: Process’ler bağımsız çalışabilir (izole), ancak bazen işbirliği yapmaları gerekir. İşbirliğinin avantajları: bilgi paylaşımı, modülerlik, computation hızlandırma. Örnek olarak Chrome tarayıcı verilir: her tab için ayrı renderer process, bir tab çökerse diğerleri etkilenmez.

İki temel IPC modeli detaylı şekilde anlatılır:

Shared Memory: İki process aynı fiziksel bellek alanı üzerinden haberleşir. Normal şartlarda her process’in bellek alanı korunur; ancak shared memory’de her iki process de okuma/yazma

yapabilir. OS sadece paylaşılan belleği tahsis eder; haberleşmenin yönetimi process'lere aittir. Avantaj: hızlı (kernel arada değil, doğrudan bellek erişimi). Dezavantaj: senkronizasyon karmaşıklığı process'lere ait.

Message Passing: Process'ler arasında mesaj aktarımı ile haberleşme. Doğrudan (direct) veya dolaylı (indirect, mailbox üzerinden) olabilir. Avantaj: distributed sistemlerde çalışır. Dezavantaj: kernel dahil olduğu için yavaş; mesaj boyutu sınırları, link kapasitesi gibi kısıtlamalar var.

Producer-Consumer problemi detaylı şekilde açıklanır. Bu problem, IPC algoritmalarını test etmek için klasik bir örnek olarak kullanılır. Buffer alanı bounded (sınırlı) olmalıdır çünkü gerçek sistemlerde sınırsız bellek yoktur. Üretici (producer) buffer'a veri ekler; tüketici (consumer) buffer'dan veri alır. Buffer dolu ise üretici bekler; buffer boş ise tüketici bekler. `in` (eklenecek index) ve `out` (çıkarılacak index) pointer'ları buffer'ı yönetir. `in == out` durumu hem "dolu" hem "boş" anlamına gelebilir; bu durumu çözmek için `count` değişkeni kullanılır.

Direct vs Indirect Communication karşılaştırması yapılır. Direct communication'da process'ler birbirini açıkça adresler (`send(P, message)`, `receive(Q, message)`); iki process arasında tipik olarak tek link kurulur. Indirect communication'da mesajlar mailbox üzerinden aktarılır; process'ler mailbox'ı paylaşır, böylece birden fazla process aynı mailbox'a yazabilir/okuyabilir.

Mailbox (Posta Kutusu) modeli detaylı açıklanır. Mailbox bir kuyruktur; mesajlar mailbox'a bırakılır, alıcı mailbox'tan alır. Mailbox'ın kapasitesi, öncelik yönetimi, mesaj sıralaması gibi tasarım kararları vardır. Kapasite sıfır ise "randevu" modeli uygulanır: gönderici ve alıcı aynı anda hazır olmalıdır. Kapasite sınırlı ise belirli sayıda mesaj kuyrukta bekleyebilir.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.